

claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-af6d734968bf0fc61.jsonl`  
- SHA-256: `613f6576d237398dcc04c4eb63ad079bf5d1390e7d08d6b53ac2fcaad81e3e86`  
- Source modified: `2026-07-07T04:13:55+00:00`  
- Imported at: `2026-07-08T16:00:26+00:00`  
- project: `wf_ddb0b3fd-b40`  
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`
```

Transcript

1. user (2026-07-07T04:11:37.203Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:

- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts):
TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

A reviewer claims this defect in the #506 diff:

- file: backend/pipeline/detectors/wasb_infer.py
- line: 1041
- summary: The GPU warp uses grid_sample padding_mode="border" (edge replication) where the cpu backend's cv2.warpAffine uses BORDER_CONSTANT black fill, so for any source whose aspect ratio differs from the 512x288 (16:9) model input the out-of-frame band content diverges between decode backends despite the code's claim to replicate cv2.warpAffine exactly.
- failure scenario: A portrait phone clip (1080x1920) run with decode_backend=nvdec_resident: scale s=max(h,w)=1920 makes the affine sample ~420 source px beyond each horizontal edge, i.e. ~44% of the model's 512-px input width. The cpu backend feeds the HRNet black side bands there (cv2.warpAffine default borderValue=0); the GPU path feeds edge-replicated smeared court/player colors instead. The model sees systematically different inputs, so detections and rally F1 can diverge far beyond the stated trajectory-level cpu-nvdec equivalence, with no warning. The cv2-parity unit test (tests/test_wasb_nvdec_resident.py, 16:9->16:9 geometry only) never samples out-of-bounds and cannot catch it. Note this exactly matches the validated reference (exp_t2_full.py also uses padding_mode="border"), and the golden gate corpus is 16:9, so only non-16:9 inputs are affected; padding_mode="zeros" would be the cv2-faithful choice.

- severity: minor

Adversarially VERIFY it: read the actual code (and the validated reference / WASB interface facts above) and try to REFUTE the claim. It is only real if the concrete failure scenario actually happens in the code as written. Default to isReal=false if the scenario cannot occur, is already guarded, or rests on a misreading.

2. user (2026-07-07T04:12:12.308Z)

```
506_diff.patch
sim_resume.py
simcache
1091 C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch
```

3. user (2026-07-07T04:12:13.464Z)

```
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_alpha-
alliv1_2026-07-06.md:84:[Omitted long matching line]
backend\config\models.py:123:      # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC
decode ? affine `grid_sample`
backend\pipeline\detectors\wasb_infer.py:39:``grid_sample`` resize replicating WASB's
``cv2.warpAffine`` exactly ? GPU ImageNet
backend\pipeline\detectors\wasb_infer.py:66:# --stream-video); "nvdec_resident" = GPU-resident
NVDEC decode + affine grid_sample
backend\pipeline\detectors\wasb_infer.py:920:# GPU-resident decode backend (#506): NVDEC decode
? affine grid_sample resize ?
backend\pipeline\detectors\wasb_infer.py:926:      """"`F.grid_sample`` sampling grid replicating
``cv2.warpAffine(img, trans,
backend\pipeline\detectors\wasb_infer.py:941:      Returns a ``[1, inp_h, inp_w, 2]`` float32 grid
for ``F.grid_sample(...,
backend\pipeline\detectors\wasb_infer.py:942:      mode='bilinear', align_corners=False,
padding_mode='border')``.
backend\pipeline\detectors\wasb_infer.py:1040:      warped = F.grid_sample(
backend\pipeline\detectors\wasb_infer.py:1041:      chw, self._grid, mode="bilinear",
align_corners=False, padding_mode="border"
backend\pipeline\detectors\wasb_infer.py:1254:      "(#506): PyNvVideoCodec NVDEC +
cv2-faithful affine grid_sample, CUDA-only, "
tests\test_wasb_nvdec_resident.py:11:affine ``grid_sample`` back to a plain resize.
tests\test_wasb_nvdec_resident.py:76:      """"grid_sample over build_affine_grid == cv2.warpAffine
on WASB's production
tests\test_wasb_nvdec_resident.py:92:      F.grid_sample(
tests\test_wasb_nvdec_resident.py:93:      chw, grid, mode="bilinear",
align_corners=False, padding_mode="border"
tests\test_wasb_nvdec_resident.py:124:      F.grid_sample(
tests\test_wasb_nvdec_resident.py:125:      chw, grid, mode="bilinear",
align_corners=False, padding_mode="border"
```

4. user (2026-07-07T04:12:22.384Z)

```
915      finally:
916          release()
917
918
919      # ----- #
920      # GPU-resident decode backend (#506): NVDEC decode ? affine grid_sample resize ?
921      # normalize ? detector, all on the GPU (no host round-trip). Validated on a real
922      # L4: ~1.7x the cv2 streaming path with rally F1 preserved (0.574 ? baseline 0.582
923      # on GX010128 at the SERVED preset).
924      # ----- #
925      def build_affine_grid(trans_inv, inp_w: int, inp_h: int, orig_w: int, orig_h: int):
926          """"`F.grid_sample`` sampling grid replicating ``cv2.warpAffine(img, trans,
927          (inp_w, inp_h), INTER_LINEAR)`` ? THE #506 lesson lives here.
928
```

```

929 The GPU resize MUST be WASB's affine warp (center/scale geometry + cv2's
930 integer-coordinates-are-pixel-centers convention), NOT a plain resize:
931 ``F.interpolate``'s half-pixel mapping shifts every sample by a constant
932 sub-pixel offset, which tips the sigmoid past ``score_threshold`` on marginal
933 frames and silently over-detects ? F1 0.58 ? 0.07 on the hard clip GX010128,
934 while the easy clip masked it at 98% agreement. Locked by a cv2-parity unit test.
935
936 ``trans_inv`` is WASB's INVERSE affine (``get_affine_transform(..., inv=1)``,
937 mapping output px ? source px; each source position is normalized to [-1, 1]
938 via ``gx = (2*x + 1)/W - 1`` (cv2's half-pixel convention). Pure torch+numpy ?
939 no WASB imports ? so the parity test runs on any CPU box.
940
941 Returns a ``[1, inp_h, inp_w, 2]`` float32 grid for ``F.grid_sample(...,
942 mode='bilinear', align_corners=False, padding_mode='border')``.
943 """
944 import numpy as np
945 import torch
946
947 ys, xs = torch.meshgrid(
948     torch.arange(inp_h, dtype=torch.float32),
949     torch.arange(inp_w, dtype=torch.float32),
950     indexing="ij",
951 )
952 out_coords = torch.stack([xs, ys, torch.ones_like(xs)], dim=-1) # [H,W,3] output px
953 mt = torch.from_numpy(np.ascontiguousarray(trans_inv, dtype=np.float32)) # [2,3]
954 src = out_coords @ mt.transpose(0, 1) # [H,W,2] source px
955 gx = (2.0 * src[..., 0] + 1.0) / orig_w - 1.0
956 gy = (2.0 * src[..., 1] + 1.0) / orig_h - 1.0
957 return torch.stack([gx, gy], dim=-1).unsqueeze(0) # [1,H,W,2]
958
959
960 def _wasb_inverse_affines(orig_h: int, orig_w: int, out_wh, out_scales):
961     """Per-scale inverse affines exactly as WASB's ``ImageDataset`` builds
962     ``trans_outputs_inv`` (``get_transform(img, wh, inv=1)``: center = frame middle,
963     scale = max(h, w), output size halved per extra scale). Returns
964     ``{scale: np.ndarray[2, 3] float64}`` ? float64 like ``cv2.getAffineTransform``,
965     matching what the DataLoader path collates for the postprocessor."""
966     import numpy as np
967     from utils.image import get_affine_transform
968
969     c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
970     s = max(orig_h, orig_w) * 1.0
971     out = {}
972     out_w, out_h = int(out_wh[0]), int(out_wh[1])
973     for scale in out_scales:
974         out[scale] = get_affine_transform(c, s, 0, [out_w, out_h], inv=1)
975         out_w //= 2
976         out_h //= 2
977     return out
978
979
980 class NvdecResidentFrontend:
981     """On-GPU frame source for the ``nvdec_resident`` detector pass (#506).
982
983     PyNvVideoCodec ``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
984     NVDEC engine; frames arrive as CUDA tensors via DLPack and never touch the host.
985     ``read_chw`` then applies the cv2-faithful affine warp (``build_affine_grid``)
986     and ImageNet-normalizes on-device, yielding the same ``[3, inp_h, inp_w]``
987     float32 tensor per frame that the ImageDataset path assembles on the CPU.
988     """
989
990     def __init__(self, video_path: str, inp_wh, gpu_id: int = 0):
991         import PyNvVideoCodec as nvc
992         import torch

```

```

994         self._torch = torch
995         self._device = torch.device("cuda", gpu_id)
996         inp_w, inp_h = int(inp_wh[0]), int(inp_wh[1])
997         try:
998             self._dec = nvc.SimpleDecoder(
999                 video_path, gpu_id=gpu_id, output_color_type=nvc.OutputColorType.RGB
1000             )
1001         except Exception as e:
1002             raise RuntimeError(
1003                 f"PyNvVideoCodec RGB SimpleDecoder failed for {video_path}: {e!r} ? "
1004                 f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
1005                 f"and libnvcuvid/libnvidia-encode on the box (deploy/cloudrun/Dockerfile
1006                 f"/ gpu_setup.sh stage them). Fall back to decode_backend=cpu."
1007             ) from e
1008         self.count = len(self._dec)
1009         probe = self._decode_rgb_hwc(0)
1010         self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
1011         trans_inp_inv = _wasb_inverse_affines(
1012             self.orig_h, self.orig_w, (inp_w, inp_h), [0]
1013         )[0]
1014         self._grid = build_affine_grid(
1015             trans_inp_inv, inp_w, inp_h, self.orig_w, self.orig_h
1016         ).to(self._device)
1017         self._mean = torch.tensor(IMAGENET_MEAN, device=self._device).view(3, 1, 1)
1018         self._std = torch.tensor(IMAGENET_STD, device=self._device).view(3, 1, 1)
1019
1020     def _decode_rgb_hwc(self, index: int):
1021         """Frame ``index`` as an on-GPU HWC RGB uint8 tensor (zero-copy via DLPack)."""
1022         torch = self._torch
1023         t = torch.from_dlpack(self._dec[index])
1024         if t.dim() == 3 and t.shape[0] == 3 and t.shape[-1] != 3:
1025             t = t.permute(1, 2, 0) # CHW layout variant ? HWC
1026         if t.dim() != 3 or t.shape[-1] != 3:
1027             raise RuntimeError(
1028                 f"unexpected NVDEC frame shape {tuple(t.shape)} for frame {index} ? "
1029                 f"expected 3-channel RGB (is output_color_type=RGB supported here?)"
1030             )
1031         return t
1032
1033     def read_chw(self, index: int):
1034         """Decoded frame ``index`` ? the normalized ``[3, inp_h, inp_w]`` float32 CUDA
1035         tensor the detector expects (warpAffine-faithful resize + ImageNet
1036         normalize)."""
1037         import torch.nn.functional as F
1038
1039         hwc = self._decode_rgb_hwc(index)
1040         chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
1041         warped = F.grid_sample(
1042             chw, self._grid, mode="bilinear", align_corners=False, padding_mode="border"
1043         ).squeeze(0)
1044         return (warped - self._mean) / self._std
1045
1046     def release(self):
1047         self._dec = None
1048
1049     def run_video_nvdec_resident(
1050         video_path: str,
1051         weights: str,
1052         out_csv: str,
1053         sport: str = "badminton",
1054         limit: int = 0,
1055         batch_size: int = 8,
1056         log_every_batches: int = 50,

```

```

1057     cache_dir: "str | None" = None,
1058     flush_every: int = 1000,
1059     fresh: bool = False,
1060 ) -> str:
1061     """GPU-resident path (#506): NVDEC-decode ``video_path`` on the GPU and feed
1062     warped+normalized frames straight to the detector ? decode, resize, and
1063     normalize never touch the host. CUDA-only by construction.
1064
1065     NOT byte-identical to the ``cpu`` decode paths (NVDEC's YUV?RGB and the GPU
1066     warp differ from cv2 at float precision) ? equivalence is trajectory-level
1067     (the #506 unit gate: 96.3% visibility agreement, median 0.036 px) and
1068     rally-F1-level (the golden gate), so the detection cache is keyed by
1069     ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
1070     cache + tracker tail as ``run()`` otherwise.
1071     """
1072     import time
1073
1074     import numpy as np

```

5. user (2026-07-07T04:12:24.219Z)

```

168-     import torch.nn.functional as F
169-     hwc = self._decode_hwc_uint8(index)
170-     chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
171-     resized = F.grid_sample(chw, self.grid, mode="bilinear",
172-                             align_corners=False, padding_mode="border").squeeze(0) #
affine-warp = cv2 match
173-     return (resized - self.mean) / self.std
174-     def stack_window(self, chw_list):
175-         return self.torch.cat(chw_list, dim=0)

```

6. user (2026-07-07T04:13:06.234Z)

```

925: def build_affine_grid(trans_inv, inp_w: int, inp_h: int, orig_w: int, orig_h: int):
955:     gx = (2.0 * src[..., 0] + 1.0) / orig_w - 1.0
960: def _wasb_inverse_affines(orig_h: int, orig_w: int, out_wh, out_scales):
969:     c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
970:     s = max(orig_h, orig_w) * 1.0
1010:     self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
1012:     self.orig_h, self.orig_w, (inp_w, inp_h), [0]
1015:     trans_inp_inv, inp_w, inp_h, self.orig_w, self.orig_h
1096:     f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
1151:     fe.orig_h, fe.orig_w, output_wh, out_scales

```

7. user (2026-07-07T04:13:06.776Z)

```

1     """GPU-resident NVDEC decode backend (#506) ? the parts testable off-GPU, plus the
2     on-box parity gate.
3
4     THE lesson this file locks (issue #506): the GPU resize MUST replicate WASB's
5     ``cv2.warpAffine(get_affine_transform(...), INTER_LINEAR)`` exactly. A naive
6     ``F.interpolate`` resize samples at half-pixel-convention positions ? a constant
7     sub-pixel shift versus cv2's integer-pixel-center affine ? which tips the sigmoid
8     past ``score_threshold`` on marginal frames and silently over-detects (rally F1
9     0.58 ? 0.07 on the hard golden clip GX010128, while the easy clip masked it at 98%
10    trajectory agreement). The grid-parity tests below fail if anyone "simplifies" the
11    affine ``grid_sample`` back to a plain resize.
12
13    The trajectory-parity contract itself (``nvdec_resident`` ? ``cpu`` on a real clip)
14    is a HARDWARE test ? it needs a CUDA GPU, PyNvVideoCodec, the WASB repo, weights and
15    a fixture clip ? so it skips everywhere else (CI is CPU-only). See
16    ``test_nvdec_resident_trajectory_matches_cpu_on_fixture`` for how to run it on the box.
17    """
18

```

```

19 import csv
20 import importlib.util
21 import os
22 import os.path as osp
23 import sys
24
25 import numpy as np
26 import pytest
27
28 from backend.pipeline.detectors import wasb_infer as wi
29
30
31 # ----- #
32 # WASB's exact affine geometry, replicated for the tests (utils/image.py
33 # ``get_affine_transform`` with rot=0: center = frame middle, scale = max(h, w),
34 # 3-point correspondence through cv2.getAffineTransform). The runtime path calls
35 # WASB's own function; the tests only need matrices of the same shape.
36 # ----- #
37 def _get_3rd_point(a, b):
38     direct = a - b
39     return b + np.array([-direct[1], direct[0]], dtype=np.float32)
40
41
42 def _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h):
43     """(forward, inverse) 2x3 matrices exactly as WASB's get_transform builds them."""
44     import cv2
45
46     c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
47     s = max(orig_h, orig_w) * 1.0
48     src = np.zeros((3, 2), dtype=np.float32)
49     dst = np.zeros((3, 2), dtype=np.float32)
50     src[0, :] = c
51     src[1, :] = c + np.array([0, -s * 0.5], dtype=np.float32)
52     dst[0, :] = [dst_w * 0.5, dst_h * 0.5]
53     dst[1, :] = dst[0, :] + np.array([0, -dst_w * 0.5], dtype=np.float32)
54     src[2, :] = _get_3rd_point(src[0, :], src[1, :])
55     dst[2, :] = _get_3rd_point(dst[0, :], dst[1, :])
56     fwd = cv2.getAffineTransform(np.float32(src), np.float32(dst))
57     inv = cv2.getAffineTransform(np.float32(dst), np.float32(src))
58     return fwd, inv
59
60
61 def _smooth_test_image(orig_w, orig_h, seed=506):
62     """Deterministic smooth float32 RGB image: smooth enough that both bilinear
63     implementations agree on identical sample positions, textured enough that a
64     sub-pixel sampling shift shows up loudly."""
65     import cv2
66
67     rng = np.random.default_rng(seed)
68     img = rng.random((orig_h, orig_w, 3)).astype(np.float32)
69     return cv2.GaussianBlur(img, (0, 0), sigmaX=2.0)
70
71
72 # ----- #
73 # The resize-fidelity lock (CPU-runnable: torch + cv2, no GPU / no WASB repo)
74 # ----- #
75 def test_affine_grid_matches_cv2_warpaffine():
76     """grid_sample over build_affine_grid == cv2.warpAffine on WASB's production
77     geometry (16:9 frame, the badminton 512x288-shaped target)."""
78     torch = pytest.importorskip("torch")
79     cv2 = pytest.importorskip("cv2")
80     import torch.nn.functional as F
81
82     orig_w, orig_h, dst_w, dst_h = 192, 108, 64, 36 # 16:9 ? 16:9, scale 3.0
83     img = _smooth_test_image(orig_w, orig_h)

```

```

84     fwd, inv = _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h)
85
86     ref = cv2.warpAffine(img, fwd, (dst_w, dst_h), flags=cv2.INTER_LINEAR)
87
88     grid = wi.build_affine_grid(inv, dst_w, dst_h, orig_w, orig_h)
89     assert tuple(grid.shape) == (1, dst_h, dst_w, 2)
90     chw = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0)
91     got = (
92         F.grid_sample(
93             chw, grid, mode="bilinear", align_corners=False, padding_mode="border"
94         )
95         .squeeze(0)
96         .permute(1, 2, 0)
97         .numpy()
98     )
99
100    diff = np.abs(got - ref)
101    # cv2 quantizes bilinear coefficients (INTER_BITS); anything beyond that noise
102    # floor means the geometry diverged. The naive-resize failure below is ~100x this.
103    assert diff.mean() < 1e-3, f"mean |?| {diff.mean():.2e} vs cv2.warpAffine"
104    assert diff.max() < 0.02, f"max |?| {diff.max():.2e} vs cv2.warpAffine"
105
106
107    def test_naive_interpolate_does_not_match_cv2_warppaffine():
108        """The #506 regression signature: F.interpolate is NOT the affine warp. If this
109        test ever fails (interpolate suddenly 'matches'), the reference setup is broken ?
110        it exists so the grid can never be quietly replaced by a plain resize."""
111        torch = pytest.importorskip("torch")
112        cv2 = pytest.importorskip("cv2")
113        import torch.nn.functional as F
114
115        orig_w, orig_h, dst_w, dst_h = 192, 108, 64, 36
116        img = _smooth_test_image(orig_w, orig_h)
117        fwd, inv = _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h)
118
119        ref = cv2.warpAffine(img, fwd, (dst_w, dst_h), flags=cv2.INTER_LINEAR)
120        chw = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0)
121
122        grid = wi.build_affine_grid(inv, dst_w, dst_h, orig_w, orig_h)
123        affine = (
124            F.grid_sample(
125                chw, grid, mode="bilinear", align_corners=False, padding_mode="border"
126            )
127            .squeeze(0)
128            .permute(1, 2, 0)
129            .numpy()
130        )
131        naive = (
132            F.interpolate(chw, size=(dst_h, dst_w), mode="bilinear", align_corners=False)
133            .squeeze(0)
134            .permute(1, 2, 0)
135            .numpy()
136        )
137
138        affine_err = np.abs(affine - ref).mean()
139        naive_err = np.abs(naive - ref).mean()
140        # The half-pixel-convention shift is a real, systematic error ? far above the
141        # affine grid's coefficient-quantization noise floor.
142        assert naive_err > 1e-3, f"naive resize unexpectedly matches cv2 ({naive_err:.2e})"
143        assert naive_err > 5 * affine_err, (
144            f"affine grid ({affine_err:.2e}) must beat naive resize ({naive_err:.2e}) "
145            f"decisively ? otherwise the #506 lesson is unlearned"
146        )
147
148

```

```

149 # ----- #
150 # Cache keying: nvdec detections are trajectory-level (not byte-level) equivalents
151 # of cpu ones, so the resumable cache must never be shared across backends.
152 # ----- #
153 def _manifest(**over):
154     m = {
155         "version": wi.CACHE_VERSION,
156         "frames_dir": "video:/x.mp4",
157         "weights": "/w.pth.tar",
158         "sport": "badminton",
159         "frames_in": 3,
160         "frames_total": 100,
161         "device": "cuda",
162     }
163     m.update(over)
164     return m
165
166
167 def test_cache_never_shared_across_decode_backends():
168     kw = dict(
169         frames_dir="video:/x.mp4",
170         weights="/w.pth.tar",
171         sport="badminton",
172         frames_in=3,
173         frames_total=100,
174         device="cuda",
175     )
176     # Pre-#506 manifests carry no decode_backend field == cpu-decoded: still valid
177     # for a cpu run (no invalidation on upgrade), NEVER for an nvdec run.
178     legacy = _manifest()
179     assert wi.manifest_compatible(legacy, **kw)
180     assert not wi.manifest_compatible(legacy, **kw, decode_backend="nvdec_resident")
181     # And an nvdec cache must not be replayed by a cpu run.
182     nv = _manifest(decode_backend="nvdec_resident")
183     assert wi.manifest_compatible(nv, **kw, decode_backend="nvdec_resident")
184     assert not wi.manifest_compatible(nv, **kw)
185
186
187 # ----- #
188 # CLI guards: the nvdec branch must fail loud BEFORE any torch/GPU work.
189 # ----- #
190 def test_cli_nvdec_resident_requires_video(monkeypatch, tmp_path):
191     monkeypatch.setattr(
192         sys,
193         "argv",
194         [
195             "wasb_infer.py",
196             "--decode-backend",
197             "nvdec_resident",
198             "--frames_dir",
199             str(tmp_path),
200             "--weights",
201             "w.pth.tar",
202             "--out",
203             str(tmp_path / "o.csv"),
204         ],
205     )
206     with pytest.raises(SystemExit, match="requires --video"):
207         wi.main()
208
209
210 def test_cli_nvdec_resident_requires_cuda_device(monkeypatch, tmp_path):
211     monkeypatch.setattr(
212         sys,
213         "argv",

```

```

214         [
215             "wasb_infer.py",
216             "--decode-backend",
217             "nvdec_resident",
218             "--video",
219             "v.mp4",
220             "--device",
221             "cpu",
222             "--weights",
223             "w.pth.tar",
224             "--out",
225             str(tmp_path / "o.csv"),
226         ],
227     )
228     with pytest.raises(SystemExit, match="cuda"):
229         wi.main()
230
231
232 def test_cli_rejects_unknown_decode_backend(monkeypatch, tmp_path, capsys):
233     monkeypatch.setattr(
234         sys,
235         "argv",
236         [
237             "wasb_infer.py",
238             "--decode-backend",
239             "gpu", # not a backend; argparse choices reject it
240             "--video",
241             "v.mp4",
242             "--weights",
243             "w.pth.tar",
244             "--out",
245             str(tmp_path / "o.csv"),
246         ],
247     )
248     with pytest.raises(SystemExit):
249         wi.main()
250     assert "invalid choice" in capsys.readouterr().err
251
252
253 # ----- #
254 # The #506 unit gate: nvdec_resident trajectory ? cpu trajectory on a real clip.
255 # HARDWARE test ? skips unless the box provides everything. Run on the GPU box
256 # (inside the torch-2.x wasb env, which has torch+cuda, PyNvVideoCodec, cv2):
257 #
258 # WASB_REPO_DIR=~/.models/WASB-SBDT \
259 # WASB_WEIGHTS=~/.models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
260 # WASB_PARITY_VIDEO=/path/to/short_fixture_clip.mp4 \
261 # python -m pytest tests/test_wasb_nvdec_resident.py -k parity -v
262 # ----- #
263 def _load_traj(path):
264     out = {}
265     with open(path, newline="") as f:
266         for row in csv.DictReader(f):
267             out[int(float(row["Frame"]))] = (
268                 int(float(row["Visibility"])),
269                 float(row["X"]),
270                 float(row["Y"]),
271             )
272     return out
273
274
275 def test_nvdec_resident_trajectory_matches_cpu_on_fixture(tmp_path):
276     repo = osp.expanduser(osp.environ.get("WASB_REPO_DIR", ""))
277     weights = osp.expanduser(osp.environ.get("WASB_WEIGHTS", ""))
278     video = osp.expanduser(osp.environ.get("WASB_PARITY_VIDEO", ""))

```

```

279     if not (repo and weights and video):
280         pytest.skip("set WASB_REPO_DIR + WASB_WEIGHTS + WASB_PARITY_VIDEO to run")
281     for p in (osp.join(repo, "src"), weights, video):
282         if not osp.exists(p):
283             pytest.skip(f"missing {p}")
284     torch = pytest.importorskip("torch")
285     if not torch.cuda.is_available():
286         pytest.skip("needs a CUDA GPU")
287     pytest.importorskip("PyNvVideoCodec")
288
289     # Stage wasb_infer.py into the WASB repo src/ (exactly as the runners do) and import
290     # THAT copy, so its WASB imports and Hydra's configs/ dir resolve.
291     import shutil
292
293     from backend.pipeline.detectors import native_wasb_runner as nwr
294
295     src = osp.join(repo, "src")
296     staged = osp.join(src, "wasb_infer.py")
297     shutil.copy(nwr._INFER_PY, staged)
298     sys.path.insert(0, src)
299     try:
300         spec = importlib.util.spec_from_file_location("wasb_infer_staged", staged)
301         mod = importlib.util.module_from_spec(spec)
302         assert spec.loader is not None
303         spec.loader.exec_module(mod)
304
305         cpu_csv = str(tmp_path / "cpu_traj.csv")
306         gpu_csv = str(tmp_path / "gpu_traj.csv")
307         mod.run_video_streaming(video, weights, cpu_csv, fresh=True)
308         mod.run_video_nvdec_resident(video, weights, gpu_csv, fresh=True)
309
310         cpu, gpu = _load_traj(cpu_csv), _load_traj(gpu_csv)
311         common = sorted(set(cpu) & set(gpu))
312         assert len(common) >= 10, "fixture too short to be a meaningful gate"
313         agree = sum(1 for f in common if cpu[f][0] == gpu[f][0]) / len(common)
314         dists = [
315             float(np.hypot(cpu[f][1] - gpu[f][1], cpu[f][2] - gpu[f][2]))
316             for f in common
317             if cpu[f][0] == 1 and gpu[f][0] == 1
318         ]
319         # Validated on the L4 (#506): 96.3% visibility agreement, median 0.036 px.
320         assert agree >= 0.95, f"visibility agreement {agree:.3f} < 0.95"
321         assert dists, "no frames visible in both trajectories"
322         assert float(np.median(dists)) <= 1.0, (
323             f"median both-visible distance {float(np.median(dists)):.3f}px > 1.0px"
324         )
325     finally:
326         sys.path.remove(src)
327

```

8. user (2026-07-07T04:13:55.230Z)

Structured output provided successfully